Resilience-Alpha — Documentation détaillée du système

*Plateforme de paiement **fictive** à reprise autonome (Zero-Touch Recovery). Réseau **fermé**, données **fictives**. Ce document explique la **logique** du système — comment on mesure, détecte, explique, remédie et vérifie — pour un jury et des coéquipiers. Il ne décrit pas comment lancer les outils. Sauf mention « illustratif », chaque affirmation correspond au comportement réel du code.*

---

0. Guide de lecture

Si vous avez 2 minutes, lisez la section 1 (le principe en une boucle de cinq temps) et le tableau de séparation des rôles : c'est l'idée directrice de tout le système.

Si vous préparez la soutenance, lisez en plus la section 7 (signatures des attaques), la section 8 (remédiation et preuve de reprise) et la section 12 (questions probables du jury).

Si vous reprenez le code, la section 6 (catalogue des KPI) et la section 11 (couverture des exigences du sujet) relient chaque composant à son rôle et à la demande qu'il satisfait.

Glossaire express

Terme	En une phrase
KPI	Indicateur chiffré de l'état de la plateforme (ex. requêtes/s), recalculé en continu.
Prometheus	L'outil qui collecte les mesures et calcule les KPI. La source unique de vérité.
WAF	Pare-feu applicatif : filtre les requêtes web à l'entrée, bloque les attaques connues.
SIEM / Loki	Centralise les journaux ; permet de retrouver l'IP d'un attaquant après coup.
kNN	Le modèle de détection : il classe l'état courant en le comparant aux cas déjà vus.
RAG	L'assistant qui *explique* l'incident en français, à partir d'une base documentaire.
Remédiation	Le moteur qui *agit* automatiquement (bloquer, redémarrer) puis *vérifie* la reprise.
Épisode	Un bloc continu de mesures d'un même état ; sert à tester le modèle honnêtement.
Zero-Touch Recovery	La reprise complète après incident, sans intervention humaine.

---

1. Vue d'ensemble

Resilience-Alpha simule une petite banque en ligne qui **se défend et se répare sans intervention humaine**. Tout le système tient dans une boucle de cinq temps :

Mesurer → Détecter → Expliquer → Remédier → Vérifier.

- **Mesurer.** Chaque service écrit des métriques et des journaux. Prometheus en tire une courte liste d'indicateurs (les *KPI*), recalculés en continu. C'est la photographie de l'état de la plateforme à l'instant présent.

- **Détecter.** Un modèle d'apprentissage (k plus proches voisins) compare cette photographie aux situations déjà observées et rend un **verdict** : trafic normal, l'un des types d'attaque connus, ou « inconnu ».
- **Expliquer.** Un assistant documentaire (RAG) rédige en français, pour l'opérateur, ce que signifie l'état détecté et ce qu'il faut faire. Il **explique seulement**.
- **Remédier.** Un moteur exécute une **action prédéfinie** adaptée au type d'incident : bloquer la source, redémarrer un service, confiner.
- **Vérifier.** La remédiation ne se déclare jamais réussie sur un simple redémarrage : elle **prouve** la reprise par une sonde de santé **et** une transaction de test qui aboutit.

La règle structurante : la séparation des rôles

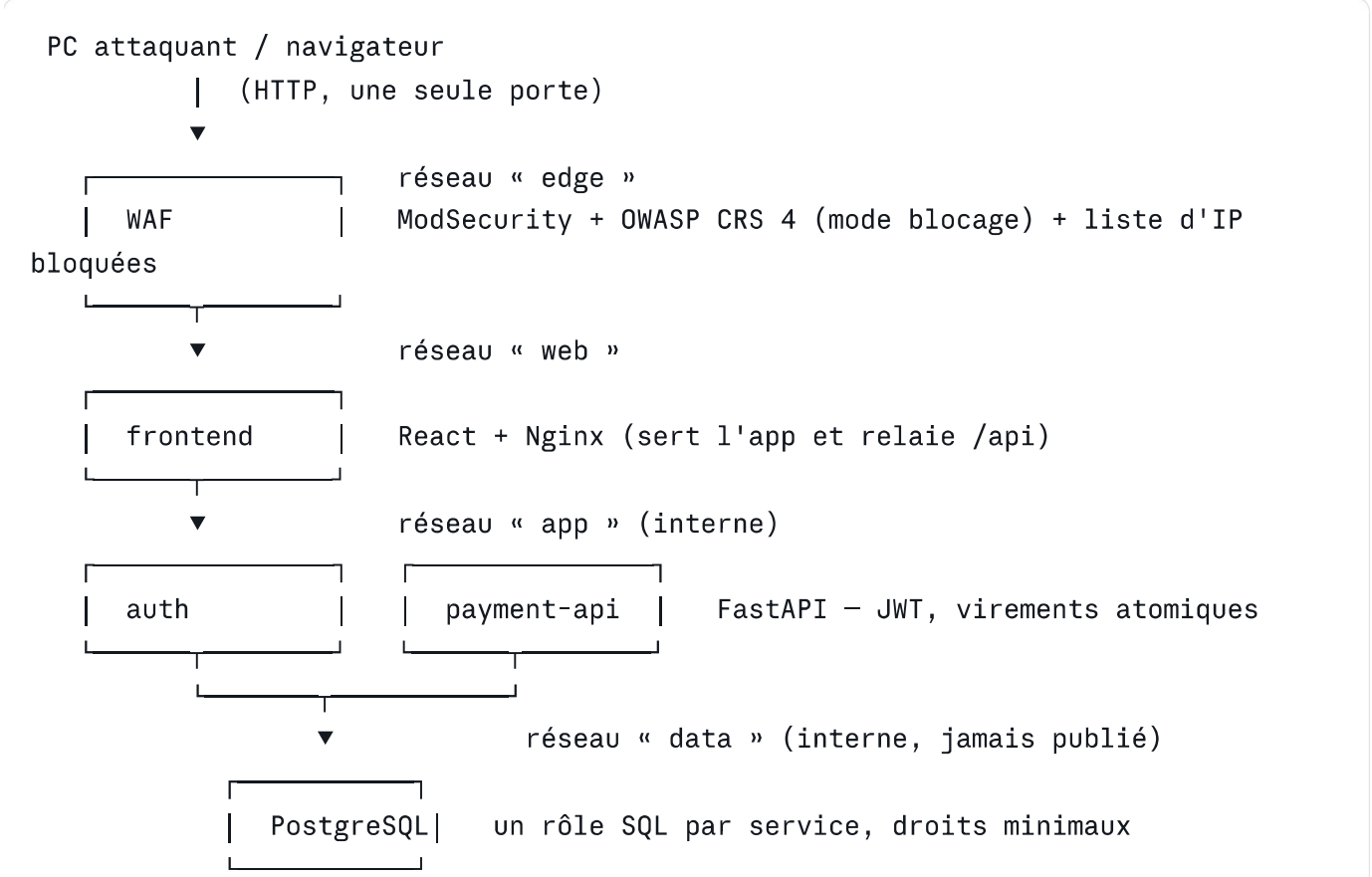
Le système sépare volontairement trois responsabilités, et aucune ne déborde sur l'autre :

Rôle	Qui	Ce qu'il fait	Ce qu'il ne fait pas
Décider	La détection (kNN)	Dit *si* c'est une attaque et *laquelle*	N'agit pas, n'explique pas
Expliquer	L'assistant RAG	Décrit l'incident et la conduite à tenir	Ne décide pas, n'agit jamais
Agir	Le moteur de remédiation	Exécute l'action prédéfinie et vérifie	N'improvise aucune action

Cette séparation est une **garantie de sûreté** : c'est la détection — déterministe et traçable — qui déclenche l'action, pas le modèle de langage. L'assistant peut se tromper dans sa prose sans jamais provoquer la moindre action sur la plateforme. C'est aussi une **exigence explicite du sujet** (§3.3 : l'assistant « ne doit pas être confondu avec l'automate chargé de la remédiation, ni disposer sans contrôle d'accès privilégiés au système »).

---

2. Architecture : les briques et qui parle à qui



Service	Rôle	Exposition
waf	Seul point d'entrée. Filtre les attaques web (CRS 4), bloque les IP listées, journalise en JSON	`127.0.0.1:8088` (ou IP réseau pour la démo)
frontend	Application web + proxy vers les API	interne
auth	Connexion, jetons JWT, verrouillage après 10 échecs	interne (métriques `:9000`)
payment-api	Comptes et virements fictifs, atomiques et idempotents	interne (métriques `:9000`)
db	PostgreSQL ; un rôle par service, droits minimaux	interne, **jamais publié**
prometheus	Collecte les métriques + calcule les **KPI** (source de vérité unique)	`127.0.0.1:9090`
loki + alloy	Centralise les journaux (SIEM), permet de retrouver l'IP d'un attaquant	interne
grafana	Tableaux de bord	`127.0.0.1:3000`

Deux idées d'architecture à retenir

Une seule porte d'entrée. Tout passe par le WAF. Les services métier (`auth`, `payment-api`) et la base ne sont jamais exposés directement. On réduit ainsi la *surface d'attaque* à un seul point, qu'on peut durcir et surveiller.

L'observabilité est en surplomb, pas sur le chemin. Prometheus, Loki et la détection n'interceptent aucune requête : ils *observent* des métriques et des journaux. Conséquence importante : la détection ne ralentit jamais les paiements, et une panne de la détection n'interrompt pas le service.

---

3. Ce qui se passe pour une requête NORMALE

1. L'utilisateur se connecte → la requête entre par le **WAF**.
2. Le WAF inspecte (règles OWASP), puis transmet au **frontend**, qui relaie vers **auth**.
3. `auth` vérifie le mot de passe (haché en **Argon2id**) et renvoie un **JWT** (jeton de session).
4. L'utilisateur fait un virement → `payment-api` vérifie le jeton, puis exécute la transaction en base de façon **atomique** : débit et crédit ensemble, ou rien. Pas d'état intermédiaire possible.
5. À chaque étape, chaque service écrit une **ligne de journal JSON** (vers Loki) et met à jour ses **métriques** (lues par Prometheus).

Cette activité normale est la **référence** du système : le détecteur apprend à quoi ressemble « tout va bien », pour pouvoir mesurer les écarts.

Pourquoi « atomique » et « idempotent » comptent

- **Atomique** : un virement interrompu en plein milieu (coupure, panne) ne laisse jamais un compte débité sans l'autre crédité. La base garantit le tout-ou-rien.
- **Idempotent** : si la même demande de virement est rejouée (réseau qui bégaie, client qui réessaie), elle n'est pas exécutée deux fois. C'est essentiel pour une plateforme de paiement, et c'est aussi ce qui permet à la **transaction de test** de la remédiation d'être rejouée sans risque.

---

4. Ce qui se passe pendant une ATTAQUE (le cycle complet)

Étape 1 — L'attaque

Depuis le PC attaquant (ou la page d'attaque locale), on joue un profil : **DDoS**, **force brute**, **injection SQL**, **scan de ports**, ou une **panne de service**. Tout vise uniquement notre hôte, sur le réseau fermé.

Étape 2 — La collecte (en continu)

Prometheus voit les KPI bouger (requêtes/s, erreurs, latence, connexions, requêtes bloquées par le WAF, échecs de connexion…). Loki enregistre les lignes de journal correspondantes.

Étape 3 — La détection (décide)

Le modèle **kNN** compare l'état courant (un vecteur de 18 KPI) aux profils connus. Il renvoie un **verdict** : l'étiquette (`ddos`, `brute_force`, `sql_injection`, `port_scan`, `service_down`, `normal`), une **confiance**, et les KPI déclencheurs. Si l'état est trop inhabituel, il répond `inconnu` → escalade humaine.

Étape 4 — L'explication (RAG — explique seulement)

L'assistant (Ollama en local, modèle ⩽ 8 B) lit la base de connaissance indexée dans FAISS et rédige, en langage clair, **ce qui se passe** et **quoi faire**. Il n'exécute **aucune** action.

Étape 5 — La remédiation (agit, puis vérifie)

Le moteur confirme l'attaque (anti-rebond : plusieurs détections d'affilée), puis exécute l'action prédéfinie du type (voir section 8), et **vérifie** la reprise.

Étape 6 — La vérification (preuve de reprise)

Le moteur refait un `/health` **et** une **transaction de test** (le compte technique `probe` vire 0,01 €). Si elle réussit → incident `resolved` ; sinon → `failed` → intervention humaine.

Étape 7 — Le journal

Chaque étape (détection → confinement → redémarrage → vérification → clôture) est écrite dans un **journal JSONL** et affichée en direct sur le tableau de bord SOC.

Déroulé illustratif d'un incident (panne de base de données)

Exemple de chronologie, pour fixer les idées. Le résultat final (~14 s, reprise automatique) est celui réellement validé ; les instants intermédiaires sont illustratifs.

t (s)	Ce qui se passe	KPI / état
0	La base est arrêtée (panne injectée)	`api_up` : 1 → **0** ; `db_errors` montent
0–5	Les virements échouent, erreurs 5xx	`tx_ok_per_sec` → **0**
~5	Nouvelle fenêtre de mesure : la signature est nette	kNN : verdict `service_down`, confiance élevée
~6	L'assistant rédige l'explication	« panne de disponibilité, redémarrer la base »
~6–10	Remédiation : redémarrage par paliers (base, puis API)	action tracée dans le journal
~10–14	Vérification : `/health` OK, puis virement `probe` de 0,01 €	la transaction **aboutit**
~14	Clôture	incident `resolved`, bandeau SOC repasse au vert

Garde-fous : actions **idempotentes** (IP déjà bloquée = rien, service déjà sain = pas de redémarrage), **délai de repos** entre deux actions du même type, **confiance minimale** requise.

---

5. Catalogue des attaques

Chaque attaque est jouée par un **harnais de laboratoire** contre la plateforme, sur un réseau fermé. Les profils utilisent **peu de connexions persistantes** et un débit borné : sur la machine de démonstration (deux cœurs), c'est le nombre de connexions simultanées qui sature, pas le débit. On produit donc la *signature* de l'attaque sans écrouler l'observabilité.

5.1 DDoS — déni de service par saturation

- **Ce que c'est.** Un volume anormal de requêtes vers un point d'entrée public (ici la sonde de santé du service de paiement), via quelques connexions maintenues ouvertes et réutilisées.
- **Ce qu'elle cherche.** Saturer l'infrastructure (WAF, API, base) pour **dégrader ou interrompre** le service des clients légitimes. Atteinte à la **disponibilité**.
- **Comment elle se manifeste.** Le WAF voit son volume de requêtes exploser ; le nombre de requêtes en cours monte ; la latence des paiements s'allonge ; le CPU applicatif augmente. Détail fidèle au code : la cible étant une sonde `/health` (exclue des compteurs applicatifs), c'est l'indicateur de **bordure** (`edge_req_per_sec`) qui porte la signature, pas le compteur de requêtes « métier ».
- **Effet observable.** Si du trafic légitime tourne en parallèle, les virements réussis par seconde chutent : le service devient inutilisable pour les vrais clients.
- **Pourquoi c'est un DoS et pas un « vrai » DDoS.** Le *D* supplémentaire de DDoS signifie *distribué* (des milliers de machines). Ici l'attaque vient d'une seule source, dans un labo fermé. L'effet mesuré (saturation, latence) et donc la **signature détectée** sont les mêmes ; seule l'échelle diffère. C'est un point à assumer clairement devant le jury.

5.2 Force brute — sur l'authentification

- **Ce que c'est.** Des tentatives de connexion répétées avec de mauvais mots de passe. Le harnais vise volontairement un compte **inexistant** (« admin »), pour produire des échecs sans verrouiller un vrai compte de démonstration.
- **Ce qu'elle cherche.** Deviner un mot de passe ou énumérer des comptes — donc **compromettre** des comptes clients.
- **Comment elle se manifeste.** Les échecs de connexion par seconde explosent, bien au-delà du bruit des fautes de frappe du trafic normal. La latence de connexion s'allonge : le hachage **Argon2** consomme du CPU à chaque tentative (c'est une protection, pas un défaut — un hachage lent ralentit l'attaquant).
- **Effet observable.** Pas d'impact sur les virements ; l'empreinte est concentrée sur l'authentification. Un verrouillage applicatif après 10 échecs limite déjà l'attaque.

5.3 Injection SQL

- **Ce que c'est.** Des requêtes contenant des charges d'injection SQL, envoyées sur un paramètre d'URL d'une API.
- **Ce qu'elle cherche.** Détourner les requêtes vers la base pour **lire ou altérer des données** — atteinte à l'**intégrité** / **confidentialité**, pas à la disponibilité.
- **Comment elle se manifeste.** Le WAF (ModSecurity, règles OWASP) **détecte et bloque** ces requêtes avec une réponse **403**, avant qu'elles n'atteignent l'application. La signature est donc le nombre de **requêtes bloquées en bordure par seconde** (`waf_blocked_per_sec`) qui monte fortement. Les requêtes de l'API étant de toute façon **paramétrées**, les erreurs de base restent basses.
- **Effet observable.** Le service continue de répondre normalement. C'est précisément le piège : l'absence d'indisponibilité ne veut pas dire qu'il ne s'est rien passé. D'où un traitement en **tentative de compromission** (voir section 8).

5.4 Scan de ports / reconnaissance

- **Ce que c'est.** Un balayage de chemins d'API inexistants (`/api/pay/…`, `/api/auth/…` suivis de chaînes aléatoires), pour cartographier la plateforme.
- **Ce qu'elle cherche.** Préparer une attaque future en découvrant les points d'entrée. Peu dommageable en soi, mais c'est un **signal d'alerte précoce**.
- **Comment elle se manifeste.** Le nombre de **404** vues par le WAF par seconde (`waf_404_per_sec`) monte. Selon la façon de se connecter, les nouvelles connexions en bordure peuvent augmenter aussi.
- **Effet observable.** Aucun impact marqué sur les virements ni sur la disponibilité.

5.5 Panne de service

- **Ce que c'est.** Un service essentiel ne répond plus. Dans la démonstration, on arrête le conteneur de la base. (Panne opérée sur l'hôte, pas via le réseau.)
- **Ce qu'elle cherche.** Représenter une défaillance — panne matérielle, épuisement de ressources, incident d'exploitation. Atteinte directe à la **disponibilité**.

- **Comment elle se manifeste.** `api_up` tombe à **0**, les erreurs de base montent, les virements réussis tombent à zéro, les 5xx augmentent. Le trafic entrant, lui, peut rester normal : c'est la plateforme qui ne **traite** plus, pas le client qui a disparu.
- **Effet observable.** Interruption nette du service, sans signal d'attaque réseau. C'est le scénario phare de la *Zero-Touch Recovery* : panne → détectée → réparée → prouvée, sans humain.

---

6. Catalogue des KPI

Les KPI sont regroupés par origine. **Source** = d'où vient la mesure brute. **Détection** = le KPI entre-t-il dans le vecteur du modèle (18 KPI) ou n'est-il que du contexte.

6.1 Trafic applicatif (services paiement et authentification)

KPI	Mesure	Source	Unité	Haut / bas signifie	Détection
`req_per_sec`	Requêtes applicatives traitées (hors `/health`)	Compteur applicatif	req/s	Haut = forte charge métier	oui
`err_4xx_per_sec`	Erreurs client (4xx) côté application	Compteur applicatif	req/s	Haut = requêtes refusées/mauvaises	oui
`err_5xx_per_sec`	Erreurs serveur (5xx) côté application	Compteur applicatif	req/s	Haut = l'application échoue (souvent panne)	oui
`inprogress`	Requêtes en cours simultanément	Jauge applicative	nombre	Haut = saturation, traitements qui s'accumulent	oui
`latency_p95_payment_seconds`	95e centile du temps de réponse des paiements	Histogramme applicatif	s	Haut = paiements lents (saturation, base en difficulté)	oui
`latency_p95_login_seconds`	95e centile du temps de réponse des connexions	Histogramme applicatif	s	Haut = connexions lentes (CPU du hachage sous force brute)	oui

6.2 Authentification

KPI	Mesure	Source	Unité	Haut / bas signifie	Détection
`login_ok_per_sec`	Connexions réussies	Compteur applicatif (auth)	req/s	Haut = activité de connexion normale	oui
`login_fail_per_sec`	Connexions en échec ou comptes verrouillés	Compteur applicatif (auth)	req/s	Haut = force brute (bien au-delà des fautes de frappe)	oui

6.3 Métier (paiement)

KPI	Mesure	Source	Unité	Haut / bas signifie	Détection
`tx_ok_per_sec`	Virements réussis	Compteur applicatif (paiement)	req/s	**Bas = service dégradé** (DDoS, panne) ; cœur du métier	oui
`tx_failed_per_sec`	Virements en échec	Compteur applicatif (paiement)	req/s	Haut = l'opération métier échoue	oui
`db_errors_per_sec`	Erreurs d'accès à la base	Compteur applicatif (paiement)	req/s	Haut = base indisponible ou en difficulté	oui

6.4 Disponibilité

KPI	Mesure	Source	Unité	Haut / bas signifie	Détection
`api_up`	1 si auth **et** paiement répondent, 0 sinon	Santé de collecte Prometheus	0 ou 1	**0 = panne** (signal le plus fort de `service_down`)	oui

6.5 Bordure (pare-feu / WAF)

KPI	Mesure	Source	Unité	Haut / bas signifie	Détection
`edge_req_per_sec`	Requêtes reçues par le WAF (toutes, `/health` compris)	Exportateur bordure	req/s	Haut = flot en entrée (signature du DDoS)	oui
`edge_connections`	Connexions actives en bordure	Exportateur bordure	nombre	Haut = nombreuses connexions ouvertes	oui
`edge_conn_per_sec`	Nouvelles connexions TCP par seconde	Exportateur bordure	conn/s	Haut = ouverture massive de connexions (scan)	oui
`waf_404_per_sec`	Réponses 404 vues par le WAF	Journaux WAF, agrégés par l'agent de logs	req/s	Haut = sondage de chemins inexistants (reconnaissance)	oui
`waf_blocked_per_sec`	Requêtes bloquées (403) par le WAF	Journaux WAF, agrégés par l'agent de logs	req/s	Haut = attaques web arrêtées en bordure (injection SQL)	oui

6.6 Ressources

KPI	Mesure	Source	Unité	Haut / bas signifie	Détection
`api_cpu_cores`	CPU des services applicatifs	Métriques de processus	cœurs	Haut = charge de calcul (saturation, hachage)	oui
`api_mem_bytes`	Mémoire des services applicatifs	Métriques de processus	octets	Contexte ; dépend du poste	non (contexte)
`host_cpu_cores`	CPU de la VM Docker entière	cAdvisor (cgroup racine)	cœurs	Contexte ; dépend du poste	non (contexte)
`host_mem_bytes`	Mémoire de la VM Docker entière	cAdvisor (cgroup racine)	octets	Contexte ; dépend du poste	non (contexte)

Note sur les sources. Les compteurs 403/404 du WAF ne sont pas des métriques natives : ils sont **reconstruits à partir des journaux JSON** du WAF par l'agent de collecte de logs, puis exposés comme métriques. Les mesures « machine entière » viennent de cAdvisor (sous Docker Desktop, la consommation **par conteneur** n'est pas disponible), c'est pourquoi on privilégie les métriques de processus et qu'on **écarte** les mesures « machine » de la détection : elles dépendent du poste et ne se généralisent pas.

Pourquoi standardiser les KPI (détail important). `edge_req_per_sec` peut valoir 200 pendant que `api_up` vaut 0 ou 1. Sans mise à l'échelle, le gros nombre « écraserait » le petit dans le calcul de distance du kNN. On ramène donc chaque KPI à une échelle commune (centré-réduit) avant de comparer : chaque indicateur pèse alors équitablement.

---

7. Signature de chaque attaque

Tableau croisé : **quels KPI bougent et dans quel sens** pour chaque état. Légende : ↑↑ forte hausse · ↑ hausse · ↓ baisse · **0** passe à zéro · — peu/pas d'effet.

KPI \ État	normal	ddos	brute_force	sql_injection	port_scan	service_down
`edge_req_per_sec`	base	↑↑	↑	↑	↑	—
`req_per_sec` (hors /health)	base	—	↑	—	↑	—
`inprogress`	bas	↑	—	—	—	—
`latency_p95_payment_seconds`	bas	↑	—	—	—	— (rien n'est traité)
`latency_p95_login_seconds`	bas	—	↑	—	—	—
`login_fail_per_sec`	~0 (fautes de frappe)	—	↑↑	—	—	—
`waf_blocked_per_sec`	~0	—	—	↑↑	—	—
`waf_404_per_sec`	~0	—	—	—	↑↑	—
`edge_conn_per_sec`	bas	—	—	—	↑	—
`tx_ok_per_sec`	normal	↓	—	—	—	**0**
`db_errors_per_sec`	~0	—	—	—	—	↑
`err_5xx_per_sec`	~0	—	—	—	—	↑
`api_up`	1	1	1	1	1	**0**
`api_cpu_cores`	bas	↑	↑	—	—	—

Comment on sépare les paires qui se ressemblent

- **DDoS vs panne.** Les deux font chuter les virements. Mais le DDoS arrive **avec** un flot en bordure (`edge_req_per_sec` ↑↑) et l'API reste debout (`api_up` = 1) ; la panne est **silencieuse en entrée** mais fait tomber `api_up` à 0 et monter les erreurs de base.
- **Force brute vs normal.** Le normal contient quelques échecs (fautes de frappe). La force brute se reconnaît à `login_fail_per_sec` **très au-dessus** de ce bruit, + latence de connexion qui s'allonge.
- **Injection SQL vs scan.** Les deux sont vues en bordure, mais l'injection déclenche des **403** (`waf_blocked_per_sec`) et le scan des **404** (`waf_404_per_sec`) : deux compteurs distincts.
- **Injection SQL vs DDoS.** L'injection ne cherche pas à saturer : `inprogress`, latence et virements restent normaux. Seul le compteur de requêtes bloquées monte.

C'est la **combinaison** de ces signaux — jamais un seul seuil — qui sépare les types. D'où le choix de l'apprentissage plutôt que de règles fixes.

---

8. La détection par apprentissage (kNN) — en détail

De l'état courant à un vecteur

À chaque instant, la plateforme est résumée par un **vecteur de 18 KPI**. On écarte volontairement les mesures « machine entière » (CPU/mémoire de la VM) : elles dépendent du poste et ne se généralisent pas.

Comment la décision est prise, pas à pas

1. **Standardisation** : chaque KPI est ramené à une échelle commune (voir section 6).
2. **Recherche des voisins** : on cherche les **k exemples les plus proches** (par défaut 15) parmi toutes les situations déjà observées et étiquetées.
3. **Vote majoritaire** : l'étiquette la plus représentée parmi ces 15 voisins gagne.

Le modèle « apprend » en **mémorisant** les exemples étiquetés ; **l'index EST le modèle**. Pas d'entraînement lourd façon réseau de neurones. C'est conforme à la contrainte du sujet (pas de création/entraînement d'un modèle « boîte noire » à partir de zéro) et cela rend la décision **transparente** : on peut toujours montrer *quels cas réels* ont pesé dans un verdict.

Exemple illustratif d'un vote

Illustratif, pour montrer le mécanisme. État courant : `edge_req_per_sec` très élevé, `inprogress` élevé, `tx_ok_per_sec` en baisse, `api_up` = 1. Les 15 voisins les plus proches dans les données : 13 étiquetés `ddos`, 1 `normal`, 1 `port_scan`. Verdict : **`ddos`**, confiance **13/15 ≈ 87 %**. KPI déclencheurs restitués : `edge_req_per_sec`, `inprogress`, `tx_ok_per_sec`.

La confiance et le cas « inconnu »

- **Confiance** : proportion des k voisins d'accord. Faible → état ambigu, à la frontière de deux situations connues.
- **Distance** : à quel point l'état est éloigné de ses voisins. Au-delà d'un **seuil** (calé sur la dispersion des données), le modèle refuse de classer de force et répond « **inconnu** ». Ce seuil est volontairement généreux : un faux « inconnu » empêcherait la remédiation de se déclencher — pire qu'une étiquette légèrement imprécise.

Le verdict fournit aussi les **indicateurs déclencheurs** (les KPI à plus de deux écarts-types de la moyenne normale, triés par importance) : ce sont eux qui rendent la décision **explicable** et qui alimentent l'assistant.

Pourquoi du ML et pas de simples seuils

- **Les seuils sont arbitraires et instables** : la bonne valeur dépend de l'heure, de la charge, du matériel. Réglé un jour, faux le lendemain.
- **Les attaques se ressemblent sur un seul KPI** : un pic de requêtes peut être un DDoS *ou* un afflux légitime. C'est la **combinaison** qui tranche.
- **Le ML raisonne sur la forme globale** : le kNN compare le vecteur entier à des cas réels déjà vus.

- **C'est maintenable** : ajouter un type d'incident = collecter des exemples étiquetés, pas réécrire une forêt de `si… alors…`.

---

9. L'assistant RAG — en détail

Son rôle : expliquer, jamais agir

Quand la détection signale un incident, l'assistant produit une **explication de 2 à 4 phrases** : ce que signifie l'état, son risque pour une plateforme de paiement, et la remédiation recommandée. Son instruction interne lui interdit de proposer d'exécuter une action et de sortir du contexte fourni. Même imprécis, il ne peut rien déclencher : **il n'a aucun accès au système**.

Le pipeline RAG, en deux temps

RAG = « génération augmentée par la recherche ». On évite de faire répondre le modèle de mémoire (risque d'invention) :

1. **Recherche.** Une **base de connaissance** (fiches décrivant chaque état : signature, enjeu, remédiation) est transformée en vecteurs et rangée dans un index de similarité (**FAISS**). À partir du type d'incident, on récupère les quelques fiches les plus pertinentes.
2. **Rédaction.** On fournit ces fiches **— et uniquement elles —** comme contexte à un modèle de langage **local** (llama3.2, 3 milliards de paramètres, via Ollama ; sous la limite de 8 B du sujet). Le modèle résume ce contexte en tenant compte des KPI déclencheurs transmis par la détection.

La base a deux origines : des **fiches écrites à la main** (une par état) et des **fiches générées automatiquement** à partir du jeu de données réel (la signature chiffrée observée). L'assistant **cite ses sources** et ne lève jamais d'erreur vers le tableau de bord : si le modèle local est indisponible, il renvoie un message clair au lieu de planter.

Pourquoi séparé de la détection

Un modèle de langage peut « halluciner ». Le laisser décider ou agir serait dangereux. En le confinant à l'explication, on a une décision **fiable et traçable** (le kNN) et une restitution **lisible** (le RAG), sans confier le pilotage de la plateforme au composant le moins prévisible.

Mise en cache (détail d'implémentation côté tableau de bord)

Un appel au modèle local prend plusieurs secondes. Pour que le tableau de bord reste fluide, l'explication n'est régénérée **que lorsque l'état change**, puis gardée en cache. La détection et les KPI, eux, se rafraîchissent en continu. On combine ainsi réactivité (règles/kNN) et profondeur d'analyse (RAG).

---

10. La remédiation automatique — en détail

La remédiation reçoit le **verdict** (type + confiance) et exécute une action **prédéfinie**. Déroulé général : **confiner → (selon la nature) agir → vérifier → journaliser.**

10.1 La logique par type

État	Nature	Action menée	Pourquoi celle-là
`service_down`	disponibilité	Redémarrage du conteneur en panne, par paliers (base puis API), puis vérification	Une panne se répare en relançant le service, mais seule une transaction réussie prouve la reprise
`ddos`	disponibilité	Arrêt de la source ; si le service reste dégradé, redémarrage de l'API, puis vérification	Couper l'origine du flot d'abord ; ne redémarrer que si nécessaire
`brute_force`	abus	Arrêt de la source ; vérification qu'un compte sain se connecte encore	Le verrouillage applicatif limite déjà l'attaque ; on confirme que le métier fonctionne
`port_scan`	reconnaissance	Arrêt de la source, surveillance ; pas de redémarrage	Peu dommageable ; on bloque et on observe
`sql_injection`	compromission	**Confinement seul** : arrêt de la source, alerte ; **jamais « résolu » automatiquement**	L'enjeu est l'intégrité, pas la disponibilité : un humain doit analyser

10.2 Comment l'attaque est réellement stoppée (production vs laboratoire)

Identifier **qui** attaque se fait via les **journaux** (Loki) : on cherche, sur une courte fenêtre, l'IP la plus active correspondant au type (beaucoup de 403 pour l'injection, de 404 pour le scan, d'échecs de connexion pour la force brute…).

- **En production.** L'IP de l'attaquant est **ajoutée à la liste de blocage du WAF**, rechargé à chaud. Ses requêtes suivantes sont rejetées en bordure (403). Attaque stoppée, sans toucher aux clients légitimes.
- **En laboratoire fermé.** L'attaque part de notre propre harnais ; son IP est une **passerelle interne protégée** qu'on ne doit jamais bloquer (sinon on coupe notre infra). La remédiation **coupe alors l'attaque à la source** (elle demande à la console d'arrêter le profil en cours). C'est pourquoi, en démo locale, l'attaque **s'arrête vraiment** après la détection.

Choix assumé : le mécanisme « réel » (blocage d'IP) reste en place et marche depuis un vrai poste attaquant ; le repli « labo » rend la démo visible sur une seule machine sans bloquer une adresse qu'on a le devoir de protéger.

10.3 La vérification de la reprise (le point crucial)

Un conteneur qui redémarre ne **prouve** rien. L'incident n'est déclaré résolu que si **deux conditions** sont réunies, avec ré-essais :

1. **Sonde de santé** : `/health` de auth **et** paiement répondent.

2. **Transaction de test** : le compte technique « probe » se connecte et effectue un **vrai virement de 0,01 €** vers un compte de collecte. La reprise n'est validée que si ce virement **aboutit**.

C'est cette transaction de bout en bout qui distingue « le service répond » de « le service fonctionne ». Tant qu'elle échoue, l'incident reste ouvert → intervention humaine.

10.4 Les garde-fous (pourquoi la remédiation n'est pas une faille)

- **Droits minimaux** : seulement des actions prédéfinies sur des cibles connues — bloquer une IP **validée** et hors liste protégée, ou redémarrer un service d'une **liste blanche**. Aucune commande arbitraire.
- **Liste « jamais bloquée »** : boucle locale, passerelles internes, hôte — la défense ne peut pas se couper elle-même.
- **Idempotence** : bloquer une IP déjà bloquée ou redémarrer un service déjà sain ne fait rien.
- **Anti-rebond + délai de repos** : on agit après plusieurs détections consécutives, et pas deux fois de suite dans un court délai.
- **Confiance minimale** : sous un seuil, on n'agit pas.
- **Injection SQL confinée, jamais résolue automatiquement** : l'intégrité des données relève d'un humain.
- **Journalisation** : chaque étape tracée, horodatée, avec son résultat ; affichée en direct.

Ce point répond directement à l'alerte du sujet (§3.4) : « une plateforme conçue pour se défendre ne doit pas introduire, par facilité, une interface d'administration accessible à n'importe quel composant ».

10.5 Le déclenchement (automatique ou humain dans la boucle)

- **Automatique** : une boucle interroge la détection en continu et agit dès qu'une attaque est confirmée (reprise autonome complète).
- **Sur action de l'opérateur** : le tableau de bord propose un bouton dès qu'une attaque est détectée ; l'opérateur confirme d'un clic, le moteur fait le reste et prouve la reprise.

---

11. Couverture des exigences du sujet

Mise en correspondance entre les demandes de l'énoncé et les composants livrés.

Exigence du sujet	Où c'est traité	Comment
§3.1–3.2 Microservices + plateforme + IaC	services/, deploy/	FastAPI (auth, payment-api), frontend, PostgreSQL, Docker Compose
§3.3 Détection par **ML** (pas de seuils en dur)	detection/	kNN sur un vecteur de 18 KPI, raccordé à la remédiation
§3.3 Assistant **RAG** (Ollama + LangChain, ⩽ 8 B)	assistant/	FAISS + llama3.2 (3 B), explique sans agir
§3.3 RAG distinct de la remédiation, sans accès privilégié	architecture	Séparation des rôles (section 1), RAG sans accès système
§3.4 Scénario d'attaque depuis un PC distinct, effet observable	labtest/, redteam/	5 profils d'attaque, signatures mesurées
§3.4 WAF (filtrage applicatif)	services/waf	ModSecurity + OWASP CRS 4 en mode blocage
§3.4 SIEM / SOC	observability (Loki + Alloy), console SOC	Journaux centralisés, dashboard SOC
§3.4 Interconnexion WAF ↔ SIEM	observability	Logs WAF → Loki → KPI 403/404 + identification d'IP
§3.4 Gestion des secrets (Vault / SOPS)	deploy/	Secrets séparés, non versionnés
§3.4 Durcissement + privilèges maîtrisés	deploy/, db	Conteneurs non-root, FS lecture seule, rôles SQL séparés
§3.4 Remédiation à droits maîtrisés	remediation/	Actions prédéfinies, listes blanches (section 10.4)
§3.5 EBIOS RM, PCA/PRA, NIS2/DORA	gouvernance (documents)	Analyse de risques et continuité reliées à cette archi
Clip chrono 60 s, reprise prouvée	démo	Panne → détection → reprise → transaction de test (section 4)

12. Défense devant le jury — questions probables

Préparez des réponses courtes et appuyées sur le système réel.

« **Votre détection, c'est juste des `if` ?** » Non : un modèle kNN sur 18 KPI standardisés. Il décide par vote des 15 cas réels les plus proches, pas par des seuils fixes. On peut montrer quels cas ont pesé dans un verdict (section 8).

« **Pourquoi pas un gros réseau de neurones ?** » Inadapté ici : peu de données, besoin de transparence et de rapidité, et le sujet interdit d'entraîner un modèle lourd de zéro. Le kNN est explicable et suffisant pour séparer les signatures.

« **Le RAG peut-il déclencher une action ?** » Jamais. Il n'a aucun accès au système ; il explique seulement. La décision vient du kNN, l'action du moteur de remédiation. C'est une exigence du sujet (§3.3) et une garantie de sûreté.

« **Comment prouvez-vous que c'est vraiment réparé ?** » Pas sur un redémarrage : par une **transaction de test** de bout en bout (virement `probe` de 0,01 €). Tant qu'elle n'aboutit pas, l'incident reste ouvert.

« **Votre DDoS n'est pas distribué.** » Exact : c'est un DoS depuis une source, en labo fermé. L'effet mesuré et la signature détectée sont les mêmes qu'un DDoS ; seule l'échelle change. Un vrai botnet est hors périmètre (et interdit).

« **Et si le moteur de remédiation est détourné ?** » Il ne fait que des actions prédéfinies sur des cibles validées (listes blanches), jamais de commande arbitraire, et ne peut pas bloquer sa propre infra. Tout est tracé (section 10.4).

« **Pourquoi l'injection SQL n'est-elle pas "résolue" automatiquement ?** » Parce que l'enjeu est l'intégrité des données, pas la disponibilité. On confine (coupe la source, alerte, conserve les journaux) et on laisse l'analyse à un humain : automatiser une « résolution » donnerait une fausse assurance.

---

13. Limites connues et pistes d'amélioration

Limites

- **Qualité de détection liée au jeu de données.** Le modèle ne reconnaît que ce qu'il a observé. Entraîné sur des attaques d'une certaine intensité puis confronté à des attaques plus douces (ou l'inverse), ses verdicts se dégradent. Une collecte suivie d'un **ré-apprentissage** sur les profils **actuels** fiabilise la détection réseau. La reprise sur **panne** ne dépend pas de l'intensité (signal franc : `api_up` = 0).
- **Machine à deux cœurs.** C'est le nombre de connexions simultanées qui sature la VM Docker, pas le débit. Attaques volontairement **modérées** : on cherche la signature, pas l'effondrement.
- **Mesure par conteneur indisponible** sous Docker Desktop : on se rabat sur les métriques de processus, et on écarte les mesures « machine » de la détection.
- **Délai de détection** : KPI moyennés sur 30 s → ~une demi-minute pour que la signature monte et que le verdict bascule.
- **Dépendance au modèle local** pour l'explication : sans lui, détection et remédiation marchent toujours, mais l'analyse en langage naturel est absente (message clair).

Pistes d'amélioration

- **Collectes régulières et nombreux épisodes** → meilleure généralisation ; mesurer la précision par type (matrice de confusion).
- **Enrichir la base de connaissance** du RAG (procédures, cas limites).

- **Signaux supplémentaires** : variété des IP sources, profils temporels, pour mieux séparer un afflux légitime d'un DDoS.
- **Passage à l'échelle** au-delà de deux cœurs : démos de DDoS plus intenses, blocage d'IP externe grandeur nature.
- **Durcir la chaîne de remédiation** : en production, passer par un intermédiaire Docker n'autorisant que « redémarrer », pour réduire encore la surface de droits.

Fin du document. La clé de lecture de tout le système tient en trois rôles séparés : ***la détection décide, l'assistant explique, la remédiation agit et vérifie.***